

module-10-debugging

Chapter 10: Debugging | 24 programs in learning order

Source: Automate the Boring Stuff with Python 1e, Al Sweigart (CC-BY-NC-SA). Syllabus Ch10: raise, traceback, assert, logging, IDLE debugger, coin-toss case study

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

Bugs are normal; debugging is a procedure. `raise Exception('msg')` triggers failures on purpose; `traceback.format_exc()` captures the crash text. `assert spam >= 0` documents assumptions - it explodes early when violated. `logging` (DEBUG/INFO/WARNING/ERROR/CRITICAL) narrates runs and can be silenced by level or disabled; log to file for long runs. The IDLE debugger steps, watches globals, and uses breakpoints to freeze time at the suspicious line.

Key ideas to carry forward

- `assert eggs.upper() == bacon.upper()` catches case mismatches immediately.
- `logging.basicConfig(level=logging.DEBUG, format=...)` once at program start.
- `logging.disable(logging.CRITICAL)` silences everything for release.
- Coin-toss program is the full case study - read it last.

Classic beginner mistakes

- ! Leaving `assert` for user-input validation (it can be switched off with `-O`).
- ! `DEBUG` spam in production - raise the level or disable.

Study order hint: 01-07 raise + traceback, 08-12 asserts, 13-19 logging, 20-24 debugger + coin-toss study.

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. assert vs exception handling?

A: assert documents programmer assumptions that must never break (disabled with -O); try/except handles expected runtime failures (bad input, network).

Q2. How do you read a traceback?

A: Bottom line is the error type/message; frames above show the call path. Top-of-stack-last - start at the bottom.

Q3. Logging levels in order?

A: DEBUG < INFO < WARNING < ERROR < CRITICAL. Set level to filter noise; disable(CRITICAL) silences all.

Q4. Why log to a file?

A: Long runs and crashes after you leave - file logs preserve evidence; console scrolls away.

Q5. Debugger vs print debugging?

A: Prints narrate; the debugger freezes time: breakpoints pause, step executes one line, watches show globals mutating.

Q6. Walk me through the coin-toss bug.

A: Random streaks look like bias; logging each flip vs expected distribution separates real bugs from probability.

Teach it to others

The one analogy that sticks

Debugging is detective work: assert is the alibi check ('the light **MUST** be green here'), logging is witness statements over time, breakpoints freeze the crime scene so you can inspect every fingerprint (variable).

What your students will get wrong

- x More prints always help (they drown signal - raise the level).
- x assert validates user input (use if/raise; asserts vanish under -O).
- x The bug is where it crashed (often upstream - trace the data back).

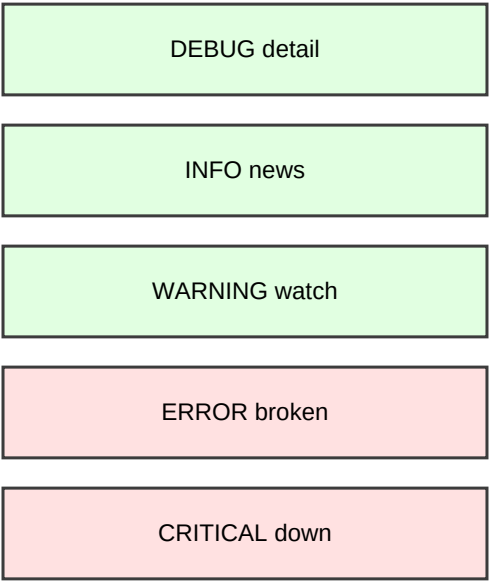
5-minute whiteboard plan

1. Trigger an assert live with a red/green light dict.
2. Add logging to the number-adder, then silence with disable.
3. Freeze the coin-toss at a breakpoint; inspect globals.
4. Exit ticket: set level so only ERROR+ shows.

Visualize it - draw these from memory

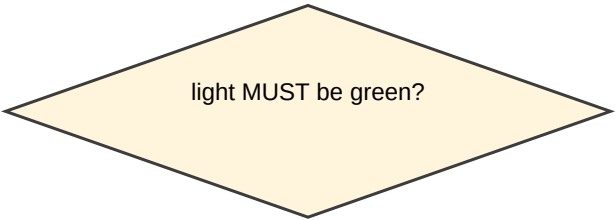
Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual 1 - logging ladder: set the bar, silence below it



Level filters noise: INFO for runs, ERROR+ for release. disable(CRITICAL) mutes everything.

Visual 2 - assert is the alibi gate



assert documents what must be true HERE. It explodes early at the real crime scene, not downstream.

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. Guarded average

```
def avg(xs):
    assert len(xs) > 0, 'need data'
    return sum(xs) / len(xs)
print(avg([2, 4]))
# EXPECTED: 3.0 (AssertionError on [])
```

X2. Logged retry

```
import logging
logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')
for i in range(3):
    logging.info('try %s', i)
print('done')
# EXPECTED: INFO lines then done
```

X3. Traceback to string

```
import traceback
try:
    1 / 0
except ZeroDivisionError:
    err = traceback.format_exc()
    print('ZeroDivisionError' in err)
# EXPECTED: True
```

Program map - all 24 at a glance

```
01 01_raising_exceptions.py
02 02_raising_exceptions_boxprint.py
03 03_raising_exceptions_oooooooooooo.py
04 04_getting_traceback_as.py
05 05_getting_traceback_as_spam.py
06 06_getting_traceback_as_2.py
07 07_getting_traceback_as_3.py
08 08_assertions.py
09 09_using_assertion_traffic.py
10 10_using_assertion_traffic_switchlights.py
11 11_using_assertion_traffic_2.py
12 12_using_assertion_traffic_switchlights_2.py
13 13_using_logging_module.py
14 14_using_logging_module_2.py
15 15_using_logging_module_debug.py
16 16_using_logging_module_debug_2.py
17 17_logging_levels.py
18 18_disabling_logging.py
19 19_logging_file.py
20 20_debugging_number_adding.py
21 21_debugging_number_adding_enter.py
22 22_breakpoints.py
23 23_breakpoints_halfway.py
24 24_debugging_coin_toss.py
```

Topic: Raising Exceptions

01/24 01_raising_exceptions.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
raise Exception('This is the error message.')  
# OUT: Traceback (most recent call last):  
# OUT:   File "<pyshell#191>", line 1, in <module>  
# OUT:     raise Exception('This is the error message.')  
# OUT: Exception: This is the error message.
```

02/24 02_raising_exceptions_boxprint.py [script]

Script: save and run; predict the output first, then compare.

```
def boxPrint(symbol, width, height):
    if len(symbol) != 1:
        raise Exception('Symbol must be a single character string.')
    if width <= 2:
        raise Exception('Width must be greater than 2.')
    if height <= 2:
        raise Exception('Height must be greater than 2.')
    print(symbol * width)
    for i in range(height - 2):
        print(symbol + (' ' * (width - 2)) + symbol)
    print(symbol * width)

for sym, w, h in (('*', 4, 4), ('0', 20, 5), ('x', 1, 3), ('ZZ', 3, 3)):
    try:
        boxPrint(sym, w, h)
    except Exception as err:
        print('An exception happened: ' + str(err))
```

03/24 03_raising_exceptions_oooooooooooo.py [script]

Script: save and run; predict the output first, then compare.

```
****
*  *
*  *
****
00000000000000000000000000000000
0          0
0          0
0          0
00000000000000000000000000000000
An exception happened: Width must be greater than 2.
An exception happened: Symbol must be a single character string.
```

Topic: Getting the Traceback as a String

04/24 04_getting_traceback_as.py [script]

Script: save and run; predict the output first, then compare.

```
def spam():  
    bacon()  
def bacon():  
    raise Exception('This is the error message.')
```



```
spam()
```

05/24 05_getting_traceback_as_spam.py [script]

Script: save and run; predict the output first, then compare.

```
Traceback (most recent call last):
  File "errorExample.py", line 7, in <module>
    spam()
  File "errorExample.py", line 2, in spam
    bacon()
  File "errorExample.py", line 5, in bacon
    raise Exception('This is the error message.')
Exception: This is the error message.
```

06/24 06_getting_traceback_as_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import traceback
try:
# OUT:         raise Exception('This is the error message.')
# OUT: except:
# OUT:         errorFile = open('errorInfo.txt', 'w')
# OUT:         errorFile.write(traceback.format_exc())
# OUT:         errorFile.close()
# OUT:         print('The traceback info was written to errorInfo.txt.')

# OUT: 116
# OUT: The traceback info was written to errorInfo.txt.
```

07/24 07_getting_traceback_as_3.py [script]

Script: save and run; predict the output first, then compare.

```
Traceback (most recent call last):
```

```
  File "<pyshell#28>", line 2, in <module>
```

```
Exception: This is the error message.
```

Topic: Assertions

08/24 08_assertions.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

podBayDoorStatus = 'open'
assert podBayDoorStatus == 'open', 'The pod bay doors need to be "open".'
podBayDoorStatus = 'I\'m sorry, Dave. I\'m afraid I can\'t do that.'
assert podBayDoorStatus == 'open', 'The pod bay doors need to be "open".'
# OUT: Traceback (most recent call last):
# OUT:   File "<pyshell#10>", line 1, in <module>
# OUT:     assert podBayDoorStatus == 'open', 'The pod bay doors need to be "open".'
# OUT: AssertionError: The pod bay doors need to be "open".
```

Topic: Using an Assertion in a Traffic Light Simulation

09/24 09_using_assertion_traffic.py [script]

Script: save and run; predict the output first, then compare.

```
market_2nd = {'ns': 'green', 'ew': 'red'}  
mission_16th = {'ns': 'red', 'ew': 'green'}
```

10/24 10_using_assertion_traffic_switchlights.py [script]

Script: save and run; predict the output first, then compare.

```
def switchLights(stoplight):
    for key in stoplight.keys():
        if stoplight[key] == 'green':
            stoplight[key] = 'yellow'
        elif stoplight[key] == 'yellow':
            stoplight[key] = 'red'
        elif stoplight[key] == 'red':
            stoplight[key] = 'green'

switchLights(market_2nd)
```

11/24 11_using_assertion_traffic_2.py [script]

Script: save and run; predict the output first, then compare.

```
assert 'red' in stoplight.values(), 'Neither light is red! ' + str(stoplight)
```

12/24 12_using_assertion_traffic_switchlights_2.py [script]

Script: save and run; predict the output first, then compare.

```
Traceback (most recent call last):
  File "carSim.py", line 14, in <module>
    switchLights(market_2nd)
  File "carSim.py", line 13, in switchLights
    assert 'red' in stoplight.values(), 'Neither light is red! ' + str(stoplight)
AssertionError: Neither light is red! {'ns': 'yellow', 'ew': 'green'}
```

Topic: Using the logging Module

13/24 13_using_logging_module.py [script]

Script: save and run; predict the output first, then compare.

```
import logging
logging.basicConfig(level=logging.DEBUG, format=' %(asctime)s - %(levelname)s
- %(message)s')
```

14/24 14_using_logging_module_2.py [script]

Script: save and run; predict the output first, then compare.

```
import logging
logging.basicConfig(level=logging.DEBUG, format='%(asctime)s - %(levelname)s
- %(message)s')
logging.debug('Start of program')

def factorial(n):
    logging.debug('Start of factorial(%s)' % (n))
    total = 1
    for i in range(n + 1):
        total *= i
        logging.debug('i is ' + str(i) + ', total is ' + str(total))
    logging.debug('End of factorial(%s)' % (n))
    return total

print(factorial(5))
logging.debug('End of program')
```

15/24 15_using_logging_module_debug.py [script]

Script: save and run; predict the output first, then compare.

```
2015-05-23 16:20:12,664 - DEBUG - Start of program
2015-05-23 16:20:12,664 - DEBUG - Start of factorial(5)
2015-05-23 16:20:12,665 - DEBUG - i is 0, total is 0
2015-05-23 16:20:12,668 - DEBUG - i is 1, total is 0
2015-05-23 16:20:12,670 - DEBUG - i is 2, total is 0
2015-05-23 16:20:12,673 - DEBUG - i is 3, total is 0
2015-05-23 16:20:12,675 - DEBUG - i is 4, total is 0
2015-05-23 16:20:12,678 - DEBUG - i is 5, total is 0
2015-05-23 16:20:12,680 - DEBUG - End of factorial(5)
0
2015-05-23 16:20:12,684 - DEBUG - End of program
```

16/24 16_using_logging_module_debug_2.py [script]

Script: save and run; predict the output first, then compare.

```
2015-05-23 17:13:40,650 - DEBUG - Start of program
2015-05-23 17:13:40,651 - DEBUG - Start of factorial(5)
2015-05-23 17:13:40,651 - DEBUG - i is 1, total is 1
2015-05-23 17:13:40,654 - DEBUG - i is 2, total is 2
2015-05-23 17:13:40,656 - DEBUG - i is 3, total is 6
2015-05-23 17:13:40,659 - DEBUG - i is 4, total is 24
2015-05-23 17:13:40,661 - DEBUG - i is 5, total is 120
2015-05-23 17:13:40,661 - DEBUG - End of factorial(5)
120
2015-05-23 17:13:40,666 - DEBUG - End of program
```

Topic: Logging Levels

17/24 17_logging_levels.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import logging
logging.basicConfig(level=logging.DEBUG, format=' %(asctime)s -
# OUT: %(levelname)s - %(message)s')
logging.debug('Some debugging details.')
# OUT: 2015-05-18 19:04:26,901 - DEBUG - Some debugging details.
logging.info('The logging module is working.')
# OUT: 2015-05-18 19:04:35,569 - INFO - The logging module is working.
logging.warning('An error message is about to be logged.')
# OUT: 2015-05-18 19:04:56,843 - WARNING - An error message is about to be logged.
logging.error('An error has occurred.')
# OUT: 2015-05-18 19:05:07,737 - ERROR - An error has occurred.
logging.critical('The program is unable to recover!')
# OUT: 2015-05-18 19:05:45,794 - CRITICAL - The program is unable to recover!
```

Topic: Disabling Logging

18/24 18_disabling_logging.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s -
# OUT: %(levelname)s - %(message)s')
logging.critical('Critical error! Critical error!')
# OUT: 2015-05-22 11:10:48,054 - CRITICAL - Critical error! Critical error!
logging.disable(logging.CRITICAL)
logging.critical('Critical error! Critical error!')
logging.error('Error! Error!')
```

Topic: Logging to a File

19/24 19_logging_file.py [script]

Script: save and run; predict the output first, then compare.

```
import logging
logging.basicConfig(filename='myProgramLog.txt', level=logging.DEBUG, format='
%(asctime)s - %(levelname)s - %(message)s')
```

Topic: Debugging a Number Adding Program

20/24 20_debugging_number_adding.py [script]

Script: save and run; predict the output first, then compare.

```
print('Enter the first number to add:')
first = input()
print('Enter the second number to add:')
second = input()
print('Enter the third number to add:')
third = input()
print('The sum is ' + first + second + third)
```

21/24 21_debugging_number_adding_enter.py [script]

Script: save and run; predict the output first, then compare.

```
Enter the first number to add:
5
Enter the second number to add:
3
Enter the third number to add:
42
The sum is 5342
```

Topic: Breakpoints

22/24 22_breakpoints.py [script]

Script: save and run; predict the output first, then compare.

```
import random
heads = 0
for i in range(1, 1001):
    if random.randint(0, 1) == 1:
        heads = heads + 1
    if i == 500:
        print('Halfway done!')
print('Heads came up ' + str(heads) + ' times.')
```

23/24 23_breakpoints_halfway.py [script]

Script: save and run; predict the output first, then compare.

```
Halfway done!
```

```
Heads came up 490 times.
```

Topic: Debugging Coin Toss

24/24 24_debugging_coin_toss.py [script]

Script: save and run; predict the output first, then compare.

```
import random
guess = ''
while guess not in ('heads', 'tails'):
    print('Guess the coin toss! Enter heads or tails:')
    guess = input()
toss = random.randint(0, 1) # 0 is tails, 1 is heads
if toss == guess:
    print('You got it!')
else:
    print('Nope! Guess again!')
    guessss = input()
    if toss == guess:
        print('You got it!')
    else:
        print('Nope. You are really bad at this game.')
```